Actividad 1: Diseñar la API del Proyecto

Introducción

Para el proyecto **Gestión de Citas Inteligente**, se diseñó la API REST del microservicio principal, correspondiente a la gestión de turnos o citas médicas. Este microservicio permite realizar las operaciones básicas de consulta, creación, actualización y eliminación de citas, siguiendo el estilo arquitectónico **REST**.

La ruta base definida para este microservicio es:

/turnos

1. Listar todos los recursos

Endpoint

GET /turnos

Descripción

Permite obtener la lista de todas las citas registradas en el sistema.

Response de ejemplo


```
[
  {
    "id": 1,
    "usuarioId": 2,
    "fecha": "2026-04-20",
    "hora": "09:00:00",
    "estado": "PENDIENTE",
    "motivo": "Consulta general"
  },
  {
    "id": 2,
    "usuarioId": 3,
    "fecha": "2026-04-21",
    "hora": "10:30:00",
    "estado": "CONFIRMADO",
    "motivo": "Control médico"
  }
]
```

2. Obtener un recurso por ID

Endpoint

GET /turnos/{id}

Ejemplo

GET /turnos/1

Descripción

Permite consultar la información de una cita específica por su identificador.

Response de ejemplo

```
{
  "id": 1,
  "usuarioId": 2,
  "fecha": "2026-04-20",
  "hora": "09:00:00",
  "estado": "PENDIENTE",
  "motivo": "Consulta general"
}
```

3. Crear un nuevo recurso

Endpoint

POST /turnos

Descripción

Permite registrar una nueva cita médica en el sistema.

Request de ejemplo

```
{
  "usuarioId": 2,
  "fecha": "2026-04-22",
  "hora": "14:00:00",
  "estado": "PENDIENTE",
  "motivo": "Odontología"
}
```

Response esperado


```
{
  "id": 3,
  "usuarioId": 2,
  "fecha": "2026-04-22",
  "hora": "14:00:00",
  "estado": "PENDIENTE",
  "motivo": "Odontología"
}
```


4. Actualizar un recurso

Endpoint

PUT /turnos/{id}

Ejemplo

PUT /turnos/3

Descripción

Permite modificar la información de una cita existente.

Request de ejemplo


```
{
  "usuarioId": 2,
  "fecha": "2026-04-23",
  "hora": "15:00:00",
  "estado": "CONFIRMADO",
  "motivo": "Odontología especializada"
}
```


Response esperado


```
{
  "id": 3,
  "usuarioId": 2,
  "fecha": "2026-04-23",
  "hora": "15:00:00",
  "estado": "CONFIRMADO",
  "motivo": "Odontología especializada"
}
```

5. Eliminar un recurso

Endpoint

DELETE /turnos/{id}

Ejemplo

DELETE /turnos/3

Descripción

Permite eliminar una cita del sistema o marcarla como eliminada, según la lógica de negocio definida.

Response esperado

```
{
  "mensaje": "Turno eliminado correctamente"
}
```

Resumen de Endpoints

Método	Endpoint	Descripción
GET	/turnos	Listar todos los turnos
GET	/turnos/{id}	Obtener un turno por ID
POST	/turnos	Crear un nuevo turno
PUT	/turnos/{id}	Actualizar un turno
DELETE	/turnos/{id}	Eliminar un turno

Conclusión Actividad 1

El diseño de esta API REST permite gestionar de forma clara y estructurada las citas médicas del sistema. Además, facilita la integración con el frontend y con otros componentes de la arquitectura distribuida.

Actividad 2: Crear Proyecto Spring Boot del Gateway

Introducción

Para centralizar el acceso a los microservicios del proyecto, se creó un **API Gateway** utilizando **Spring Boot** y **Spring Cloud Gateway**.
Este componente actúa como punto de entrada único al sistema, permitiendo enrutar las solicitudes del cliente hacia el microservicio correspondiente.

Paso 1: Creación del proyecto en Spring Initializr

Se ingresó a la plataforma:

https://start.spring.io

Y se configuró el proyecto con los siguientes parámetros:

- **Project:** Maven
- **Language:** Java
- **Spring Boot:** versión compatible
- **Java:** 17
- **Dependency:** Spring Cloud Gateway

Paso 2: Configuración del archivo application.yml

Se configuró el archivo application.yml del gateway para enrutar las solicitudes al microservicio de turnos.

Ejemplo de configuración

```yaml
server:
  port: 8080

spring:
  application:
    name: api-gateway

  cloud:
    gateway:
      routes:
        - id: turnos-service
          uri: http://localhost:8082
```

```
predicates:
  - Path=/turnos/**
```

Explicación de la configuración

- El gateway escucha en el puerto **8080**
- Las solicitudes que lleguen a la ruta /turnos/** se redirigen al microservicio **turnos-service**
- El microservicio de turnos escucha en el puerto **8082**

De esta forma, el cliente no accede directamente al microservicio, sino que siempre pasa primero por el gateway.

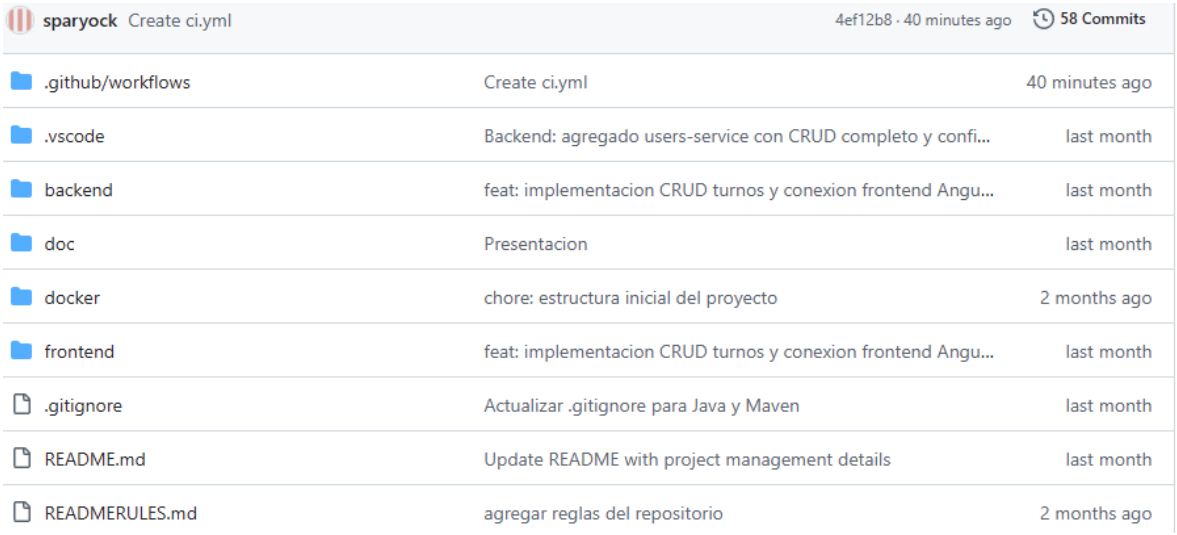
Paso 3: Integración al monorepo

El proyecto del gateway fue agregado al monorepo del equipo en la carpeta correspondiente.
Aunque en la guía se menciona gateway/, en el proyecto real se encuentra adaptado como:

api-gateway/

Esto mantiene coherencia con la estructura real del sistema.

Estructura esperada



sparyock Create ci.yml	4ef12b8 · 40 minutes ago	58 Commits
.github/workflows	Create ci.yml	40 minutes ago
.vscode	Backend: agregado users-service con CRUD completo y confi...	last month
backend	feat: implementacion CRUD turnos y conexion frontend Angu...	last month
doc	Presentacion	last month
docker	chore: estructura inicial del proyecto	2 months ago
frontend	feat: implementacion CRUD turnos y conexion frontend Angu...	last month
.gitignore	Actualizar .gitignore para Java y Maven	last month
README.md	Update README with project management details	last month
READMERULES.md	agregar reglas del repositorio	2 months ago

Conclusión Actividad 2

La creación del API Gateway permitió definir un punto central de acceso al sistema, mejorando la organización de la arquitectura y facilitando la comunicación entre cliente y microservicios.

Actividad 3: Probar con Postman / Thunder Client

Introducción

Para validar el funcionamiento de los endpoints diseñados, se utilizó una herramienta de prueba de APIs como **Postman** o **Thunder Client**.
Estas herramientas permiten enviar solicitudes HTTP, visualizar respuestas y documentar el comportamiento de la API de manera práctica.

Paso 1: Instalación de la herramienta

Se instaló una herramienta de pruebas de API, ya sea:

- **Postman**
- **Thunder Client** en Visual Studio Code

Estas herramientas permiten probar solicitudes GET, POST, PUT y DELETE fácilmente.

Paso 2: Creación de la colección

Se creó una colección llamada, por ejemplo:

API Gestión de Citas Inteligente

Dentro de esta colección se agregaron los endpoints definidos en la Actividad 1:

- GET /turnos
- GET /turnos/{id}
- POST /turnos
- PUT /turnos/{id}
- DELETE /turnos/{id}

Paso 3: Documentación de requests y responses

Cada request fue documentado con su body de ejemplo y la respuesta esperada.

Ejemplo de request POST

Método: POST
URL:

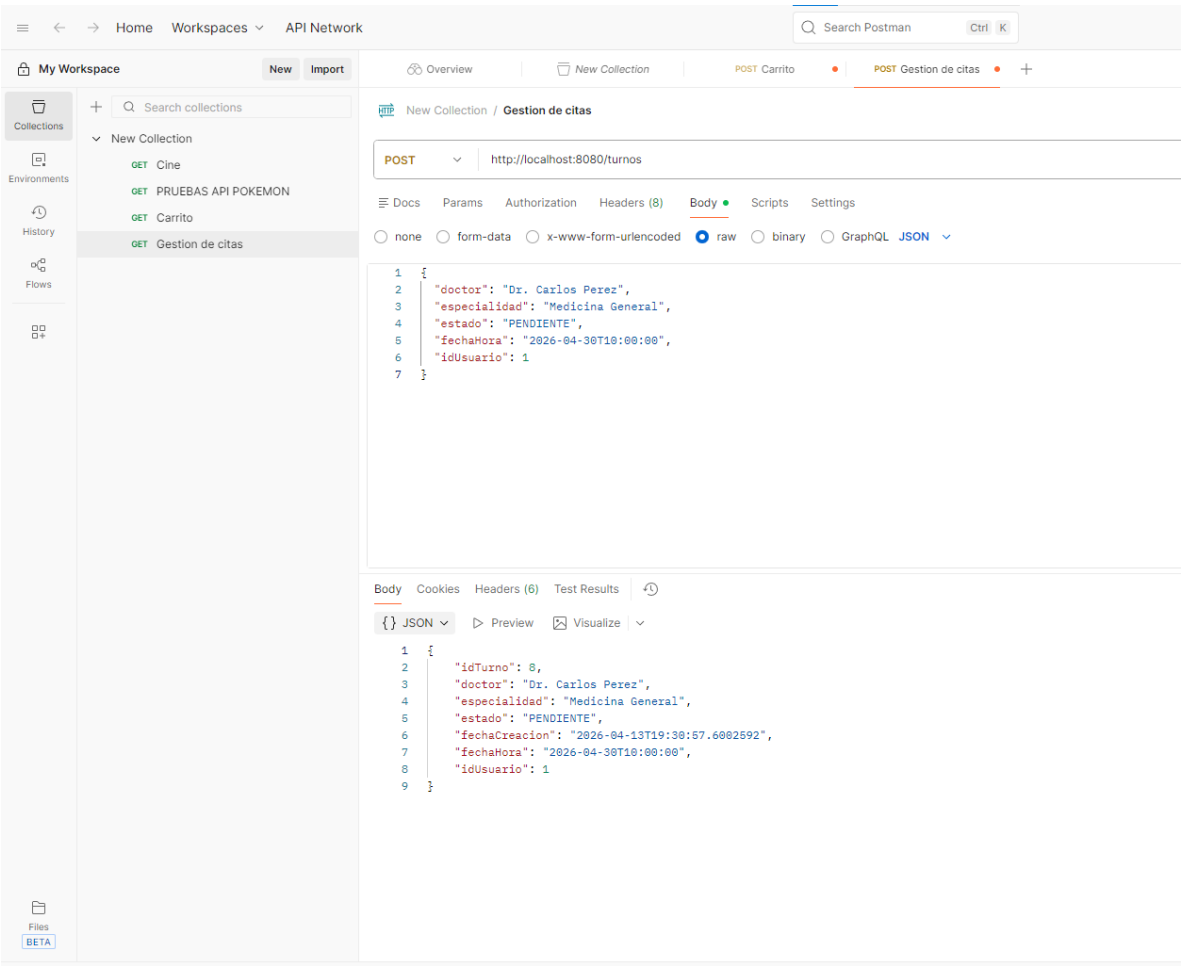
http://localhost:8080/turnos

Body:

```
{
  "usuarioId": 2,
  "fecha": "2026-04-22",
  "hora": "14:00:00",
  "estado": "PENDIENTE",
  "motivo": "Odontología"
}
```

Response esperado:

```
{
  "id": 3,
  "usuarioId": 2,
  "fecha": "2026-04-22",
  "hora": "14:00:00",
  "estado": "PENDIENTE",
  "motivo": "Odontología"
}
```



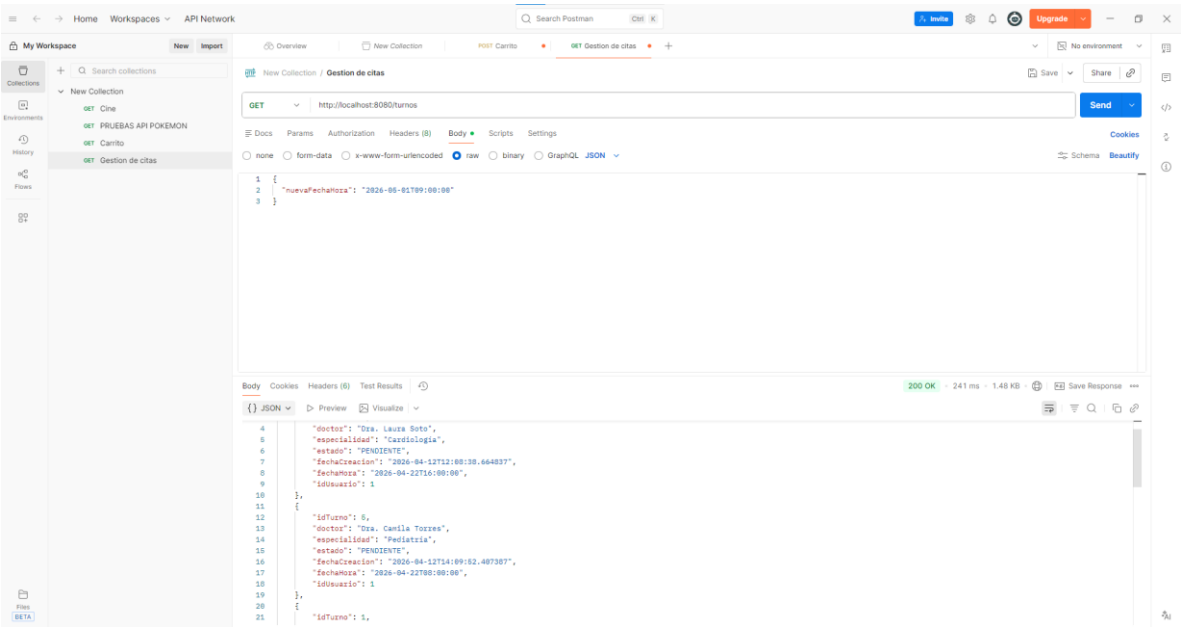
Ejemplo de request GET

Método: GET
URL:

http://localhost:8080/turnos/1

Response esperado:


```
{
  "id": 1,
  "usuarioId": 2,
  "fecha": "2026-04-20",
  "hora": "09:00:00",
  "estado": "PENDIENTE",
  "motivo": "Consulta general"
}
```



Importancia de la colección

La colección creada en Postman o Thunder Client funciona como una **documentación viva de la API**, ya que permite:

- Probar rápidamente los endpoints
- Compartir ejemplos con el equipo
- Validar respuestas
- Detectar errores de integración
- Tener una referencia clara del funcionamiento del sistema

Conclusión Actividad 3

El uso de Postman o Thunder Client permitió validar y documentar el comportamiento de la API del sistema, facilitando las pruebas del backend y la integración con el frontend.

Reflexión Final

¿Qué sucede si el API Gateway se cae? ¿Es un single point of failure?

Si el API Gateway se cae, los clientes no pueden acceder fácilmente a los microservicios, porque este componente actúa como punto central de entrada al sistema.

Por esta razón, sí puede convertirse en un **single point of failure** si solo existe una instancia.

Para evitar este problema, en ambientes reales se suelen usar varias instancias del gateway, balanceadores de carga y mecanismos de alta disponibilidad.

¿Cuáles son las ventajas de REST sobre otros protocolos como gRPC o GraphQL?

REST tiene varias ventajas, especialmente en proyectos académicos y MVPs:

- Es fácil de entender e implementar
- Usa HTTP estándar
- Se integra fácilmente con navegadores, frontend y herramientas como Postman
- Permite trabajar rápidamente con JSON
- Tiene una curva de aprendizaje más simple

Aunque gRPC puede ser más rápido y GraphQL más flexible, REST resulta más práctico para proyectos como Gestión de Citas Inteligente.

¿Cómo cambia la estrategia de Service Discovery cuando usas Docker vs. máquinas manuales?

Cuando se usan **máquinas manuales**, normalmente los servicios se conectan por **IP y puerto**, por ejemplo localhost:8082.

En cambio, cuando se usa **Docker Compose**, los contenedores pueden comunicarse usando directamente el **nombre del servicio**, por ejemplo:

turnos-service:8082

Esto facilita la comunicación entre microservicios y hace que la arquitectura sea más ordenada y portable.